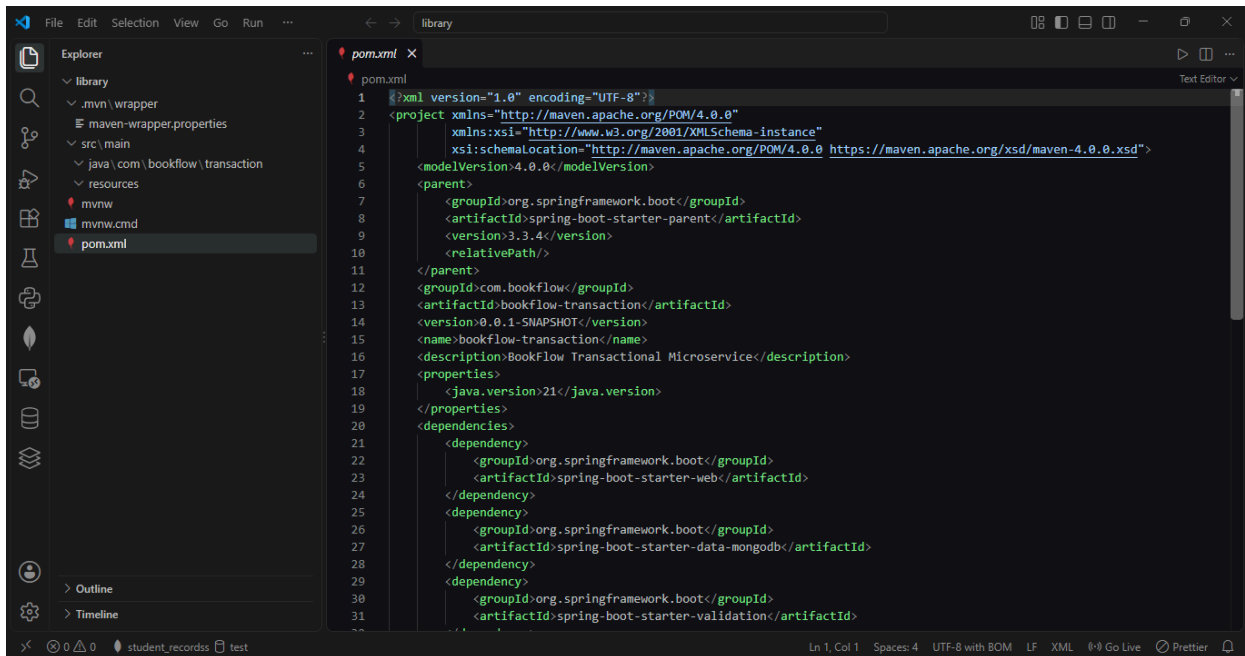
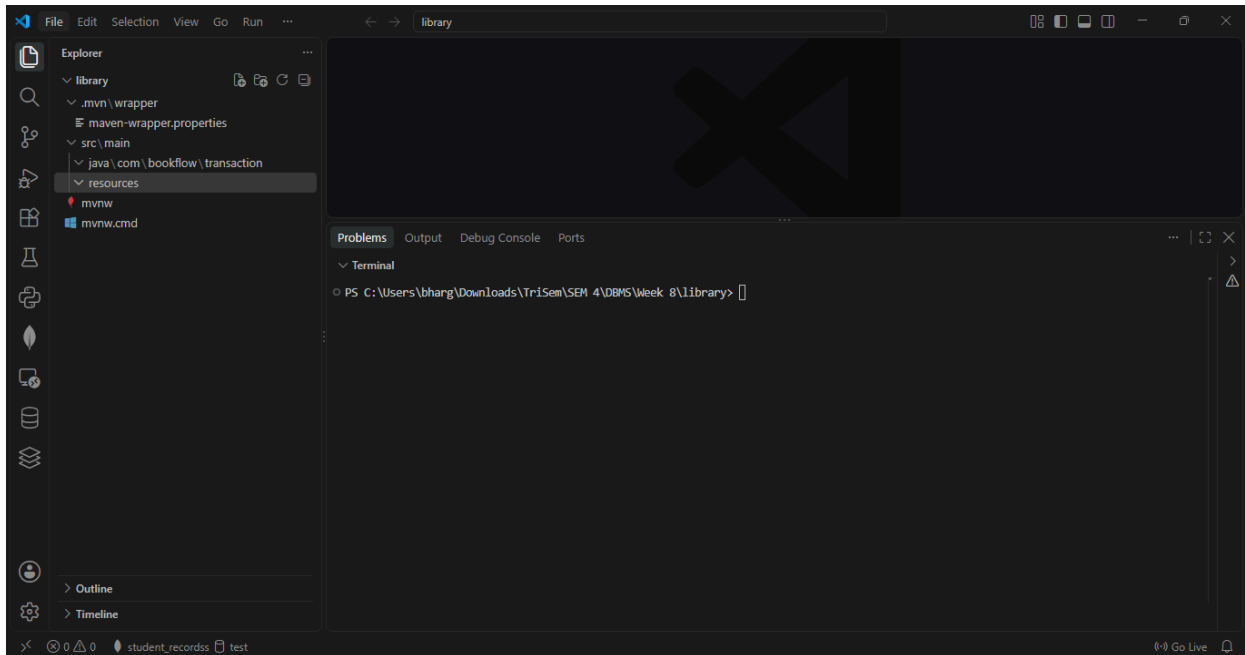
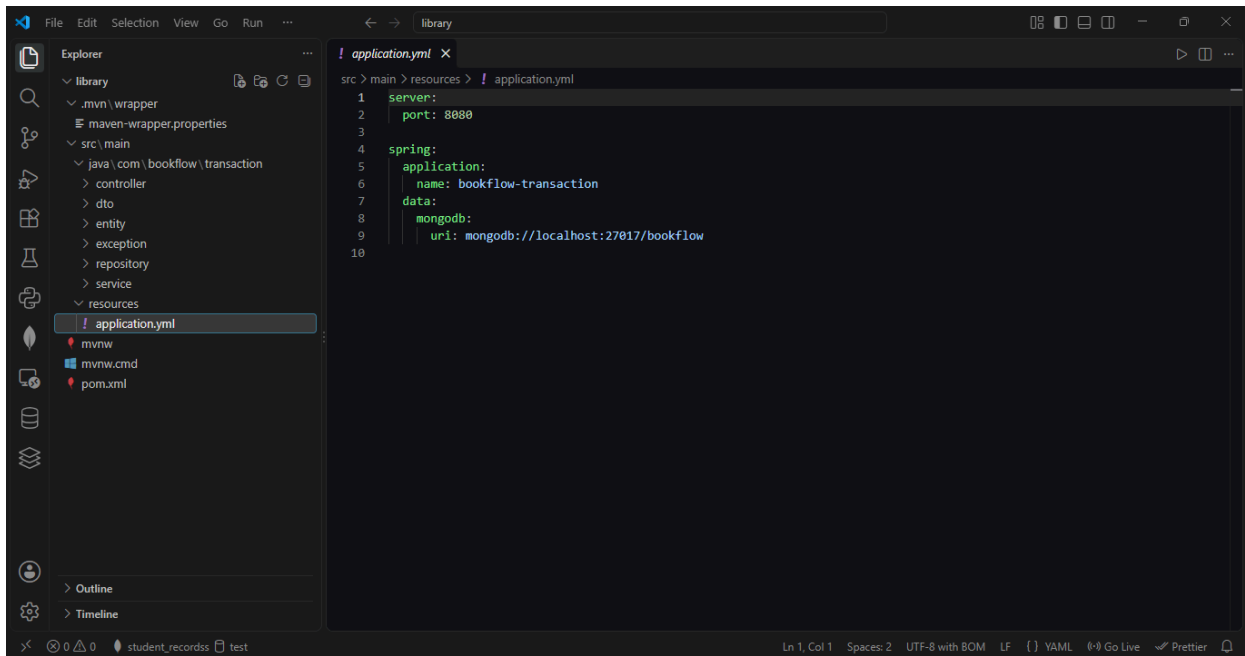
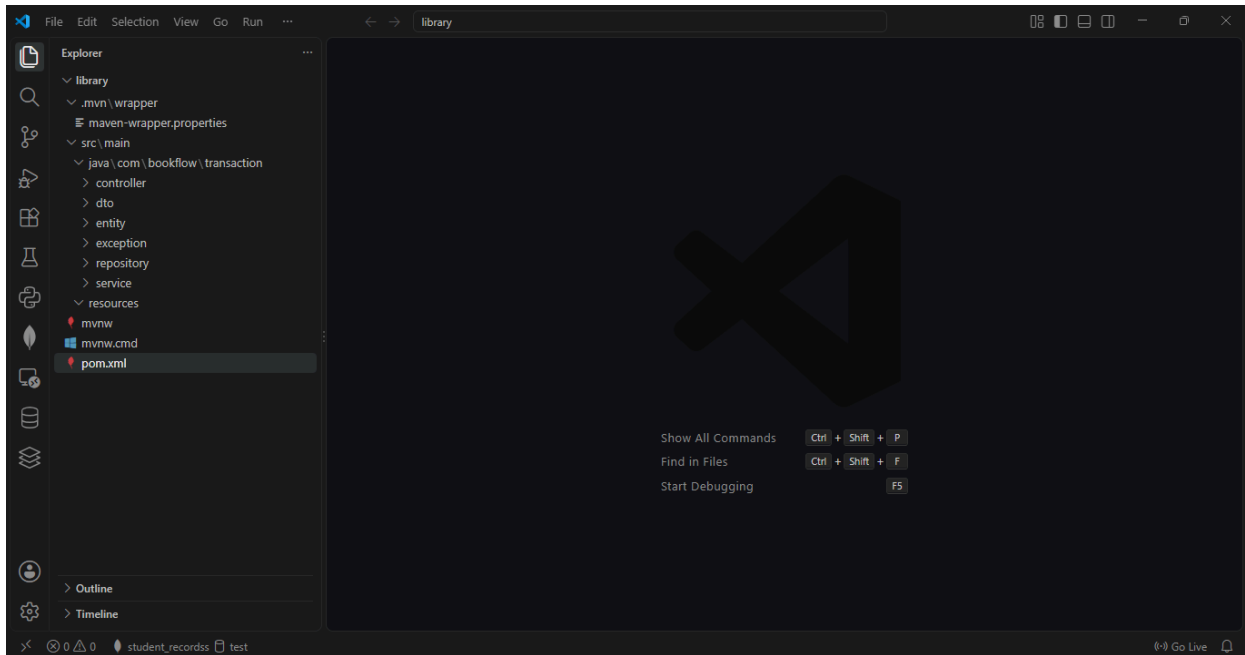
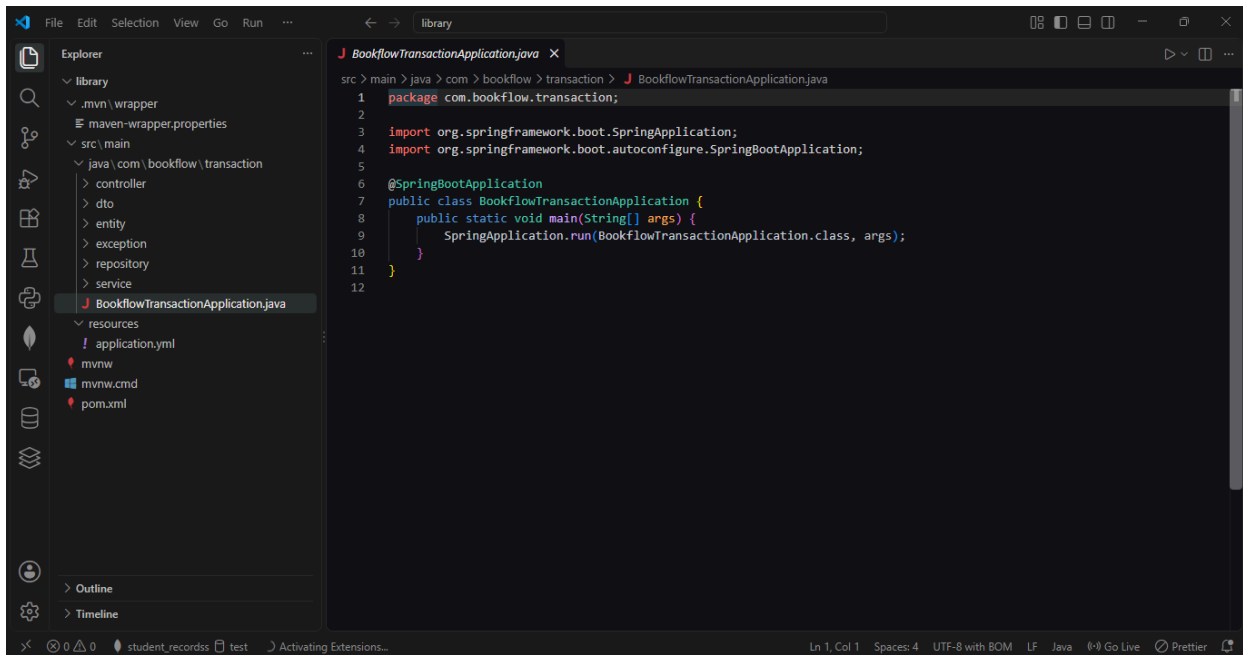
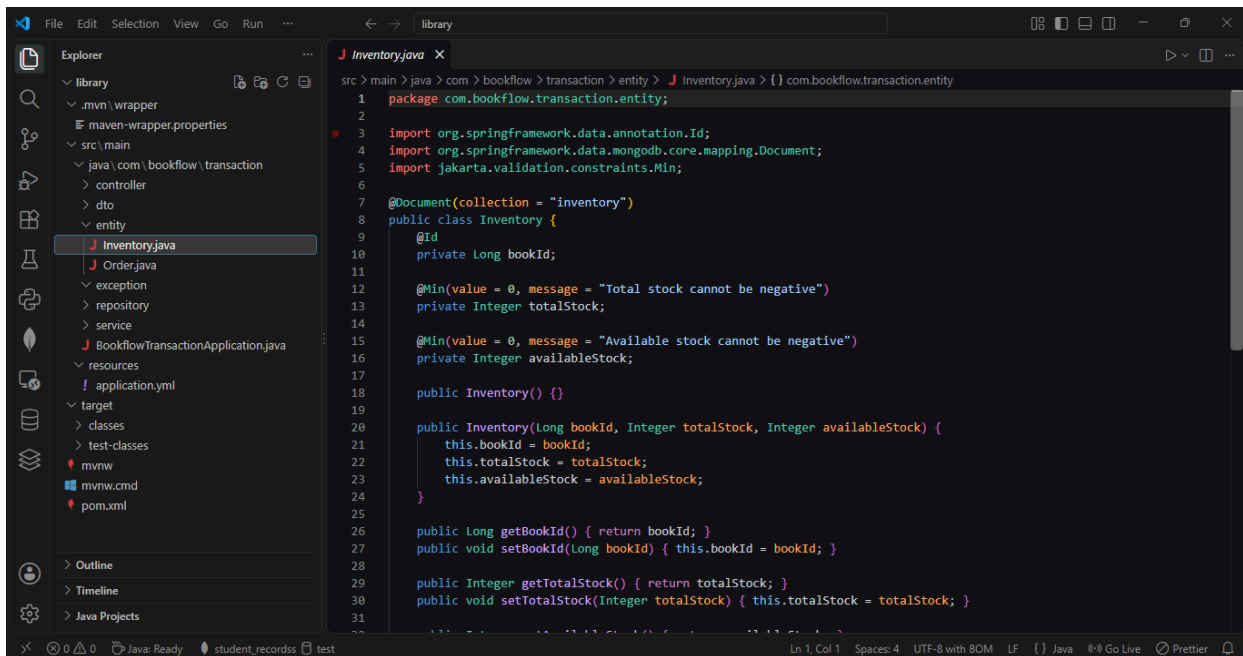








```
src > main > java > com > bookflow > transaction > J BookflowTransactionApplication.java
1 package com.bookflow.transaction;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class BookflowTransactionApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(BookflowTransactionApplication.class, args);
10    }
11 }
12
```



```
src > main > java > com > bookflow > transaction > entity > J Inventory.java > {} com.bookflow.transaction.entity
1 package com.bookflow.transaction.entity;
2
3 import org.springframework.data.annotation.Id;
4 import org.springframework.data.mongodb.core.mapping.Document;
5 import jakarta.validation.constraints.Min;
6
7 @Document(collection = "inventory")
8 public class Inventory {
9     @Id
10    private Long bookId;
11
12    @Min(value = 0, message = "Total stock cannot be negative")
13    private Integer totalStock;
14
15    @Min(value = 0, message = "Available stock cannot be negative")
16    private Integer availableStock;
17
18    public Inventory() {}
19
20    public Inventory(Long bookId, Integer totalStock, Integer availableStock) {
21        this.bookId = bookId;
22        this.totalStock = totalStock;
23        this.availableStock = availableStock;
24    }
25
26    public Long getBookId() { return bookId; }
27    public void setBookId(Long bookId) { this.bookId = bookId; }
28
29    public Integer getTotalStock() { return totalStock; }
30    public void setTotalStock(Integer totalStock) { this.totalStock = totalStock; }
31
32 }
```

```
src > main > java > com > bookflow > transaction > dto > J OrderRequest.java > {} com.bookflow.transaction.dto
1 package com.bookflow.transaction.dto;
2
3 import jakarta.validation.constraints.NotNull;
4
5 public class OrderRequest {
6     @NotNull(message = "userId is required")
7     private Long userId;
8
9     @NotNull(message = "bookId is required")
10    private Long bookId;
11
12    public OrderRequest() {}
13
14    public OrderRequest(Long userId, Long bookId) {
15        this.userId = userId;
16        this.bookId = bookId;
17    }
18
19    public Long getUserId() { return userId; }
20    public void setUserId(Long userId) { this.userId = userId; }
21
22    public Long getBookId() { return bookId; }
23    public void setBookId(Long bookId) { this.bookId = bookId; }
24 }
25
```

```
src > main > java > com > bookflow > transaction > exception > J BookOutOfStockException.java > {} com.bookflow.transaction.exception
1 package com.bookflow.transaction.exception;
2
3 public class BookOutOfStockException extends RuntimeException {
4     public BookOutOfStockException(String message) {
5         super(message);
6     }
7 }
8
```

```
src > main > java > com > bookflow > transaction > service > J InventoryService.java > {} com.bookflow.transaction.service
1 package com.bookflow.transaction.service;
2
3 import com.bookflow.transaction.entity.Inventory;
4 import com.bookflow.transaction.repository.InventoryRepository;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 public class InventoryService {
9     private final InventoryRepository inventoryRepository;
10
11     public InventoryService(InventoryRepository inventoryRepository) {
12         this.inventoryRepository = inventoryRepository;
13     }
14
15     public Inventory getInventory(Long bookId) {
16         return inventoryRepository.findById(bookId)
17             .orElseThrow(() -> new RuntimeException(message: "Book not found"));
18     }
19
20     public Inventory saveInventory(Inventory inventory) {
21         return inventoryRepository.save(inventory);
22     }
23 }
24
```

```
src > main > java > com > bookflow > transaction > repository > J InventoryRepository.java > {} com.bookflow.transaction.repository
1 package com.bookflow.transaction.repository;
2
3 import com.bookflow.transaction.entity.Inventory;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 public interface InventoryRepository extends MongoRepository<Inventory, Long> {
7 }
8
```



```
PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\library> Invoke-RestMethod -Uri "http://localhost:8080/orders" -Method POST
● -ContentType "application/json" -Body '{"userId": 501, "bookId": 101}' | ConvertTo-Json
{
  "id": "6abccb5fd00d605dd4e90de5",
  "userId": 501,
  "bookId": 101,
  "status": "BORROWED"
}
○ PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\library> █
```

```
PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\library> Invoke-RestMethod -Uri "http://localhost:8080/inventory/101" -Method GET | ConvertTo-Json
{
  "bookId": 101,
  "totalStock": 5,
  "availableStock": 4
}
○ PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\library> █
```

```
PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\library> try {
>>   Invoke-RestMethod -Uri "http://localhost:8080/orders" -Method POST -ContentType "application/json" -Body '{"userId": 502,
"bookId": 103}'
>> } catch {
>>   $stream = $_.Exception.Response.GetResponseStream()
>>   $reader = New-Object System.IO.StreamReader($stream)
>>   $reader.ReadToEnd()
>> }
{"error":"OUT_OF_STOCK","status":409,"message":"Book 103 is out of stock"}
PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\library> █
```

Service Library 8.3.7

- admin
- bookflow 96K
 - Query
 - Collections (2)
 - inventory
 - Columns
 - _id Number
 - totalStock Number
 - availableStock Number
 - _class String
 - Index
 - orders
 - Columns
 - _id Object ID
 - userid Number
 - bookid Number
 - status String
 - _class String
 - Index
 - config
 - local
 - studentDB
 - test

Properties DATA bookflow

```
db('bookflow').collection('inventory').find({}).limit(100).toArray()
```

Search Results

	_id Number	totalStock Number	availableStock Number	_class String
>	101	5	4	com.bookflow.transaction.e
>	102	3	3	com.bookflow.transaction.e
>	103	2	0	com.bookflow.transaction.e

Cost: 3ms < 1 > Total 3

Service Library 8.3.7

- admin
- bookflow 96K
 - Query
 - Collections (2)
 - inventory
 - Columns
 - _id Number
 - totalStock Number
 - availableStock Number
 - _class String
 - Index
 - orders
 - Columns
 - _id Object ID
 - userid Number
 - bookid Number
 - status String
 - _class String
 - Index
 - config
 - local
 - studentDB
 - test

Properties DATA bookflow

```
db('bookflow').collection('orders').find({}).limit(100).toArray()
```

Search Results

	_id Object ID	userid Number	bookid Number	status String	_class String
>	ObjectID('6abccb5fd00d605	501	101	BORROWED	com.bookflow.transaction.e

Cost: 2ms < 1 > Total 1